

Protocol Independent Notification Connector - Final Raporu

Bu proje, farklı protokollerden gelen bildirimleri tek bir ortak formata dönüştüren, Docker Compose ile uçtan uca çalışabilen protokol bağımsız bir connector sistemidir. Sistem Redis, RabbitMQ, WebSocket ve Webhook kaynaklarından veri alarak Backend API'ye iletmekte ve React tabanlı canlı dashboard üzerinde göstermektedir.

Kullanılan Teknolojiler

- .NET 8
- ASP.NET Core Minimal API
- .NET Worker Service
- React + Vite
- RabbitMQ 3 Management
- Redis 7
- Docker & Docker Compose

Sistem Mimarisi

Simulator -> RabbitMQ / Redis / WebSocket / Webhook -> Connector -> Backend API -> React Frontend

Docker Servisleri ve Portlar

- Frontend: http://localhost:3000
- Backend API: http://localhost:8080
- WebSocket Server: http://localhost:5050
- RabbitMQ AMQP: localhost:5672
- RabbitMQ Management: http://localhost:15672
- Redis: localhost:6379

Container isimleri (docker compose ps)

- notificationconnector-backend-1
- notificationconnector-connector-1
- notificationconnector-frontend-1
- notificationconnector-rabbitmq-1
- notificationconnector-simulator-1
- notificationconnector-websocketserver-1
- redis

Sistemi Aya ğ a Kald ırma Komutlar ı

Projeyi klonlay ın:

```
git clone https://github.com/bredkeklol/NotificationConnector.git  
cd NotificationConnector
```

T ü m sistemi derleyip ba ş lat ın:

```
docker compose up -d --build
```

Servisleri kontrol edin:

```
docker compose ps
```

Connector loglar ı n ı izleyin:

```
docker compose logs -f connector
```

Sistemi durdurun:

```
docker compose down
```

Test Komutlar ı

Redis testi:

```
docker exec -it redis redis-cli PUBLISH notifications "hello redis"
```

RabbitMQ testi:

```
docker compose exec rabbitmq rabbitmqadmin publish exchange=amq.default routing_key=notifications  
payload="hello rabbitmq"
```

Webhook testi (PowerShell):

```
Invoke-RestMethod -Method Post -Uri http://localhost:8080/webhook -ContentType "application/json"  
-Body '{"title":"Webhook","message":"Hello"}
```

Ç al ı ş ma Prensibi

Her kaynak i ç in ayr ı bir ISourceAdapter implementasyonu bulunmaktad ır. Adapter'lar gelen ham mesaj ı RawMessage olarak ü retir. ConnectorCore bu mesajlar ı normalize ederek NotificationEnvelope format ı na d ö n ü ş t ü r ü r ve Backend API'ye g ö nderir. Frontend belirli aral ı klarla API'den bildirimleri ç ekerek canl ı listeyi g ü nceller.

Tamamlanan Ö zellikler

- WebSocket adapter
- Redis adapter
- RabbitMQ adapter
- Webhook adapter
- Configuration ile adapter seçimi
- Docker Compose ile tek komut kurulum
- React canlı dashboard
- Hata toleransı ve try/catch yönetimi
- Merkezi loglama

Sonuç

Proje kapsamında dört farklı protokolden veri alabilen, bu verileri ortak bir şemaya dönüştüren, Backend API üzerinden saklayıp React arayüzünde canlı olarak gösterebilen uçtan uca dockerize bir sistem başarıyla geliştirilmiştir. Sistem tek komutla ayağa kalkmakta, farklı adapter'lar yapılandırılma üzerinden etkinleştirilebilmekte ve teslim gereksinimlerini karşılamaktadır.